

# PayFlow API - Spring Boot Assignment Write-Up

*REST API for user registration, wallet balance, and transaction recording*

**GitHub Repository:** <https://github.com/rohit-munde/-PayFlow>

Submitted by: Rohit Munde

## Submission Summary

PayFlow is implemented as a database-backed Spring Boot REST API. The project uses Spring Web, Spring Data JPA, H2 database, validation, global exception handling, and JWT-based authentication. Because the base project already had an authentication system, user registration is exposed through /auth/register and the PayFlow APIs are protected behind JWT authentication. This keeps the API closer to a production-style fintech backend while still satisfying the required user and transaction workflows.

- Users can be registered with full name, email, password, UPI ID, phone number, and opening wallet balance.
- Users can be listed, fetched by ID, fetched by UPI ID, and filtered by balance using a JPQL @Query.
- Transactions can be recorded from one UPI ID to another without foreign-key relationships, as required.
- All required APIs were tested through Postman and data persistence was verified in H2 Console.

## Task 1 - Project Setup

The project is a Spring Boot backend application. It uses Spring Web to expose REST APIs, Spring Data JPA for repository-based persistence, and H2 as the in-memory database used for development and assignment testing. The application follows a layered structure with entity, repository, service, controller, dto, exception, config, and filter packages. The assignment specifically asks for entity, repository, service, and controller packages, and those layers are present in the implementation.

| Layer / Package | Purpose                                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------------------------------|
| entity          | Contains JPA entity classes such as User and Transaction. These classes are mapped to database tables.               |
| repository      | Contains Spring Data JPA repositories such as UserRepository and TransactionRepository.                              |
| service         | Contains business logic such as user lookup, UPI lookup, and transaction recording.                                  |
| controller      | Contains REST controllers that expose API endpoints for authentication, users, and transactions.                     |
| dto             | Contains request/response DTOs to avoid exposing sensitive fields such as password and internal security properties. |
| exception       | Contains ApiResponse and custom exceptions used by global exception handling.                                        |
| filter/config   | Contains JWT authentication filter and security configuration for protected APIs.                                    |

## Spring Boot Features Used in PayFlow

- Embedded server: The application runs on embedded Tomcat, so no external server installation is required. Running the Spring Boot application starts the REST API immediately on localhost:8080.
- Auto-configuration: Spring Boot automatically configures Spring MVC, Jackson JSON conversion, Spring Data JPA, H2 datasource, Hibernate, validation, and repository beans based on the dependencies present in pom.xml.
- Production-ready defaults: The project uses standard conventions such as application.properties configuration, structured package scanning, validation support, global exception handling, and secure API access using JWT.

## How to Run the Application

```
git clone https://github.com/rohit-munde/-PayFlow
cd -PayFlow
mvn clean install
mvn spring-boot:run
```

Base URL used during testing:

`http://localhost:8080/api`

H2 Console URL used during testing:

`http://localhost:8080/api/h2-console`

## Task 2 - Entities and Database

Two JPA entities are used: User and Transaction. The User entity represents a registered PayFlow wallet user and also integrates with the existing JWT authentication flow. The Transaction entity records a money transfer between two UPI IDs. As required, the transaction does not use @ManyToOne or foreign-key relationships; it stores senderUpild and receiverUpild as plain strings.

### User Entity Fields

- id - auto-generated primary key inherited from BaseEntity
- fullName - user full name
- email - unique email used for login
- password - encrypted and write-only
- role - security role, default USER
- upild - unique UPI ID used for PayFlow lookup
- phoneNumber - phone number of the user
- balance - wallet opening/current balance
- createdAt, updatedAt, isActive, isDeleted - inherited audit/status fields

### Transaction Entity Fields

- id - auto-generated primary key inherited from BaseEntity
- senderUpild - UPI ID of sender
- receiverUpild - UPI ID of receiver
- amount - transaction amount
- createdAt, updatedAt, isActive, isDeleted - inherited audit/status fields

## H2 and JPA Configuration

```
spring.datasource.url=jdbc:h2:mem:payflowdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

## Representative Hibernate DDL Generated by JPA

The project does not use hand-written SQL for creating tables. Hibernate creates/updates tables from the JPA entity mappings. A representative generated DDL for the two assignment tables is shown below; the exact ordering of columns can vary depending on Hibernate and inherited BaseEntity fields.

```
create table users (
    id bigint generated by default as identity,
```

```

created_at timestamp(6),
created_by varchar(255),
is_active boolean,
is_deleted boolean,
updated_at timestamp(6),
updated_by varchar(255),
balance numeric(12,2),
email varchar(150) not null unique,
full_name varchar(340) not null,
password varchar(255) not null,
phone_number varchar(15),
role varchar(255),
upi_id varchar(100) unique,
primary key (id)
);

create table transactions (
    id bigint generated by default as identity,
    created_at timestamp(6),
    created_by varchar(255),
    is_active boolean,
    is_deleted boolean,
    updated_at timestamp(6),
    updated_by varchar(255),
    amount numeric(12,2) not null,
    receiver_upi_id varchar(100) not null,
    sender_upi_id varchar(100) not null,
    primary key (id)
);

```

The screenshot shows the H2 Console interface with two SQL queries and their results. The first query, 'SELECT \* FROM USERS;', returns two rows of user data. The second query, 'SELECT \* FROM TRANSACTIONS;', returns one row of transaction data.

| ID | CREATED_AT                 | CREATED_BY | IS_ACTIVE | IS_DELETED | UPDATED_AT                 | UPDATED_BY | BALANCE   | EMAIL           | FULL_NAME    | PASSWORD                                                         | PHONE_NUMBER | ROLE | UPI_ID       |
|----|----------------------------|------------|-----------|------------|----------------------------|------------|-----------|-----------------|--------------|------------------------------------------------------------------|--------------|------|--------------|
| 1  | 2026-05-31 00:48:21.469857 | null       | TRUE      | FALSE      | 2026-05-31 00:48:21.469882 | null       | 3000.00   | priya2@test.com | Priya Sharma | \$2a\$11\$CSHgFo7W.LQmht0lpkcywsiONhgio1FDzw4VeBoj4e.4XQ5qzkj22  | 9876543211   | USER | priya@okaxis |
| 2  | 2026-05-31 00:55:26.715058 | null       | TRUE      | FALSE      | 2026-05-31 00:55:26.715081 | null       | 100000.00 | rohit@test.com  | Rohit Munde  | \$2a\$11\$3ztMmZ9ZUavdqmMteJNNpOxILMDdQKqj97bZB8wS5eRwKNzYK1czu6 | 9175073243   | USER | rohit@okaxis |

| ID | CREATED_AT                 | CREATED_BY | IS_ACTIVE | IS_DELETED | UPDATED_AT                | UPDATED_BY | AMOUNT | RECEIVER_UPI_ID | SENDER_UPI_ID |
|----|----------------------------|------------|-----------|------------|---------------------------|------------|--------|-----------------|---------------|
| 1  | 2026-05-31 00:55:31.360816 | null       | TRUE      | FALSE      | 2026-05-31 00:55:31.36075 | null       | 500.00 | rohit@okaxis    | priya@okaxis  |

*Screenshot: H2 Console showing persisted USERS and TRANSACTIONS records.*

## Task 3 - Repository Layer

The repository layer uses Spring Data JPA. `UserRepository` extends `JpaRepository<User, Long>`, which provides built-in CRUD operations such as `save`, `findById`, `findAll`, and `delete`. `TransactionRepository` extends `JpaRepository<Transaction, Long>`, which provides persistence operations for transaction records.

```

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    Optional<User> findByUpiId(String upiId);

    @Query("SELECT u FROM User u WHERE u.balance > :amount")
    List<User> findUsersWithBalanceGreaterThan(BigDecimal amount);
}

public interface TransactionRepository extends JpaRepository<Transaction, Long> {
}

```

### Derived Query: findByUpild(String upild)

The method findByUpild is a derived query method. Spring Data JPA reads the method name, understands that the field after findBy is upild, and generates a query against the upi\_id column. No SQL needs to be written manually for this lookup.

```

select u1_0.id, u1_0.balance, u1_0.email, u1_0.full_name, u1_0.password,
       u1_0.phone_number, u1_0.role, u1_0.upi_id
from users u1_0
where u1_0.upi_id = ?

```

The ? placeholder means Hibernate prepares the SQL with a parameter instead of directly concatenating the UPI ID string into the query. At runtime, the value passed to findByUpild(String upild) is safely bound to that placeholder. This improves safety, avoids manual SQL construction, and helps prevent SQL injection patterns.

## Task 4 - Service Layer

The service layer contains application logic and sits between controllers and repositories. UserService handles creating users, fetching all users, finding users by ID, finding users by UPI ID, and executing the balance-above custom query. TransactionService validates the sender/receiver UPI IDs and saves the transaction record. Balance deduction was intentionally not implemented because the assignment says that the transaction service should only persist the transaction record for now.

```

public User getPayFlowUserById(Long id) {
    return userRepository.findById(id)
        .orElseThrow(() -> new UserNotFoundException(String.valueOf(id)));
}

public User findPayFlowUserByUpiId(String upiId) {
    return userRepository.findByUpiId(upiId)
        .orElseThrow(() -> new UpiIdNotFoundException(upiId));
}

public Transaction sendMoney(Transaction transaction) {
    if (transaction.getSenderUpiId().equals(transaction.getReceiverUpiId())) {
        throw new InvalidTransactionException("Sender and receiver UPI ID cannot be same");
    }
    userRepository.findByUpiId(transaction.getSenderUpiId())
        .orElseThrow(() -> new UpiIdNotFoundException(transaction.getSenderUpiId()));
    userRepository.findByUpiId(transaction.getReceiverUpiId())
        .orElseThrow(() -> new UpiIdNotFoundException(transaction.getReceiverUpiId()));
    return transactionRepository.save(transaction);
}

```

### Dependency Injection Explanation

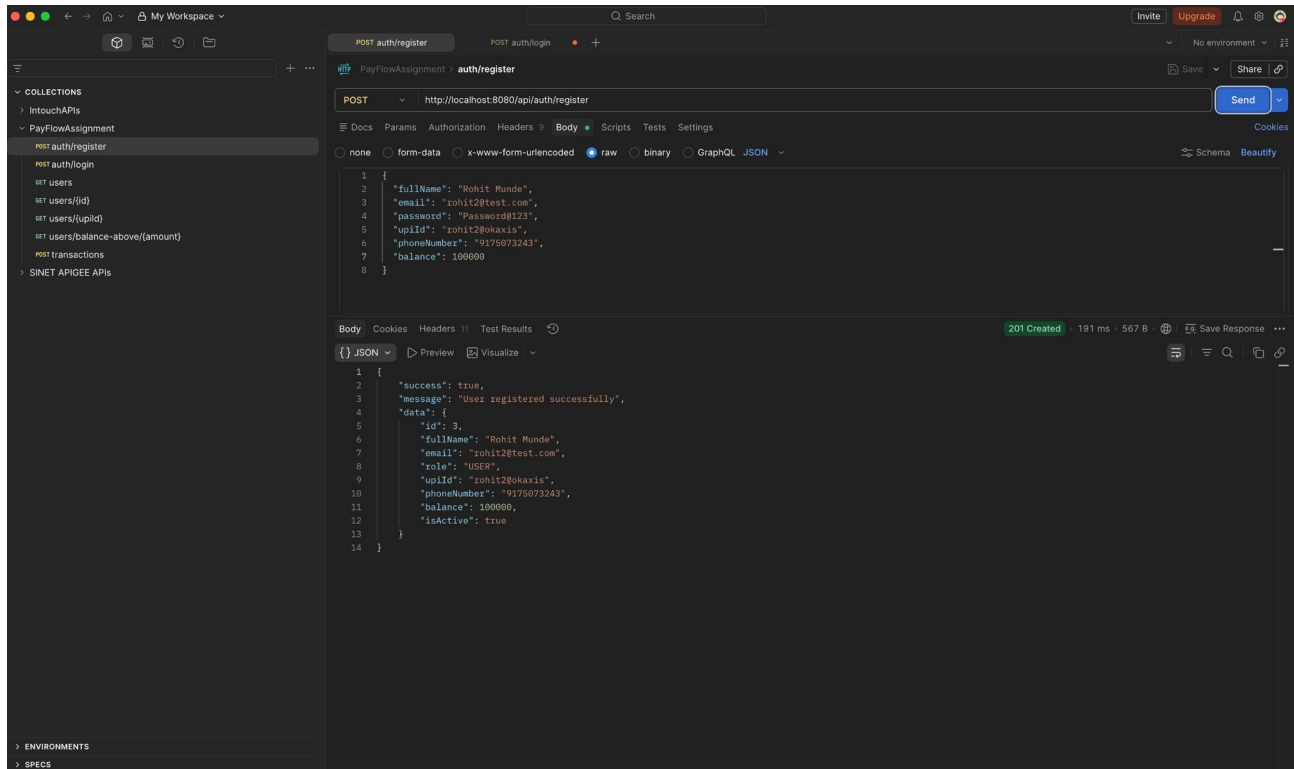
Repositories are injected into services using constructor injection. During application startup, Spring scans classes annotated with @Repository, @Service, and @RestController. It creates beans for repositories and services and automatically provides required dependencies through the constructor. Because of this, the service does not manually create repository objects using new; Spring manages the object lifecycle and wiring.

## Task 5 - Controllers and REST Endpoints

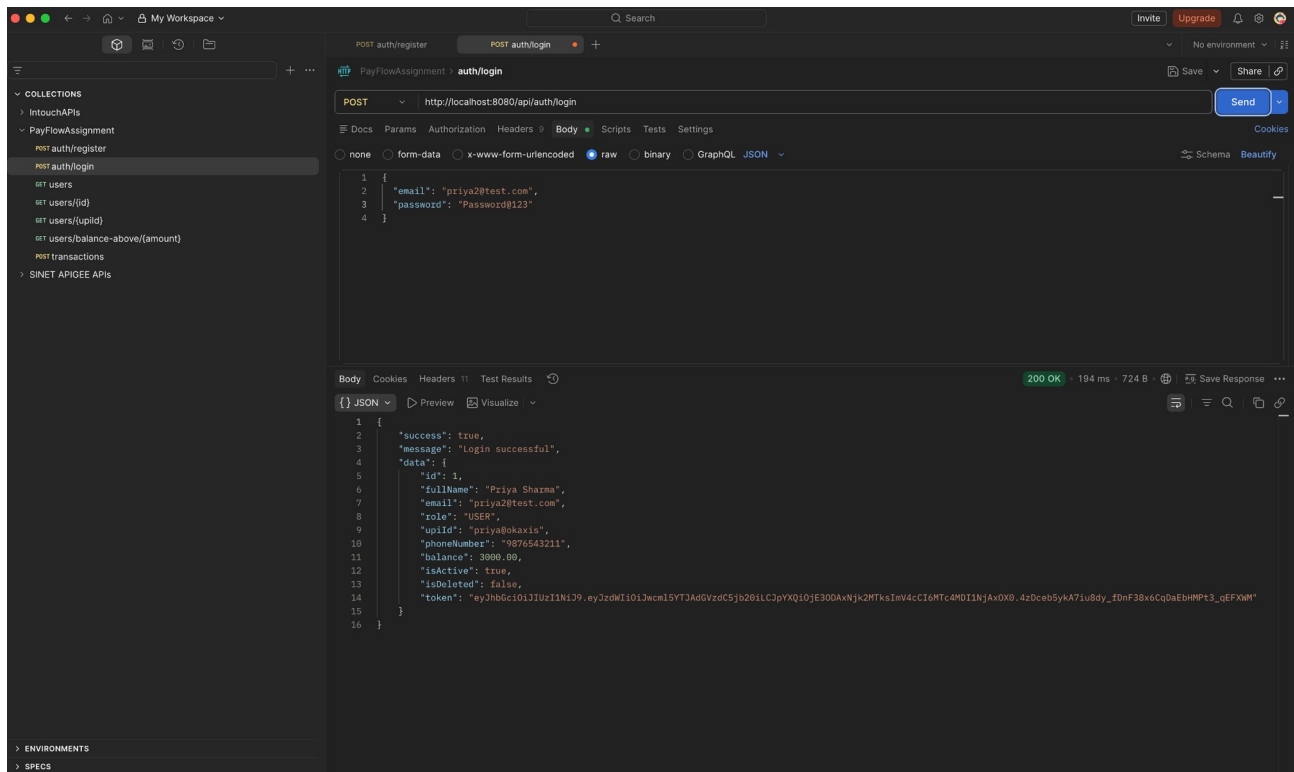
The project exposes authentication APIs through AuthController and PayFlow APIs through PayFlowUserController and TransactionController. Since the base backend already had JWT authentication, /auth/register and /auth/login are public, while /users and /transactions endpoints are protected and require a valid Bearer token.

| Method | Endpoint                      | Auth         | Purpose                                                 |
|--------|-------------------------------|--------------|---------------------------------------------------------|
| POST   | /auth/register                | Public       | Register a PayFlow user with UPI ID and opening balance |
| POST   | /auth/login                   | Public       | Login and receive JWT token                             |
| GET    | /users                        | Bearer token | List all registered users                               |
| GET    | /users/{id}                   | Bearer token | Look up user by ID                                      |
| GET    | /users/upi/{upid}             | Bearer token | Look up user by UPI ID                                  |
| GET    | /users/balance-above/{amount} | Bearer token | Return users whose balance is above a given amount      |
| POST   | /transactions                 | Bearer token | Record a money transfer between two UPI IDs             |

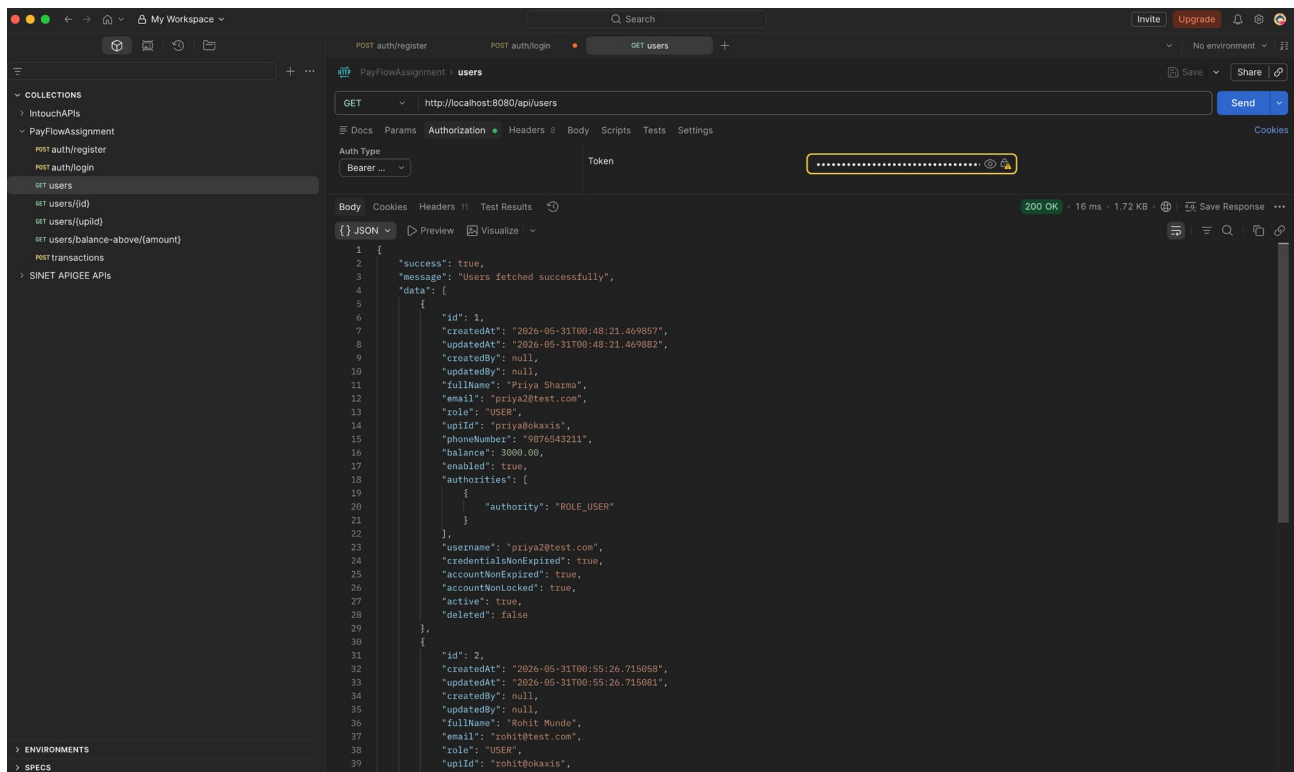
### Postman API Testing Screenshots



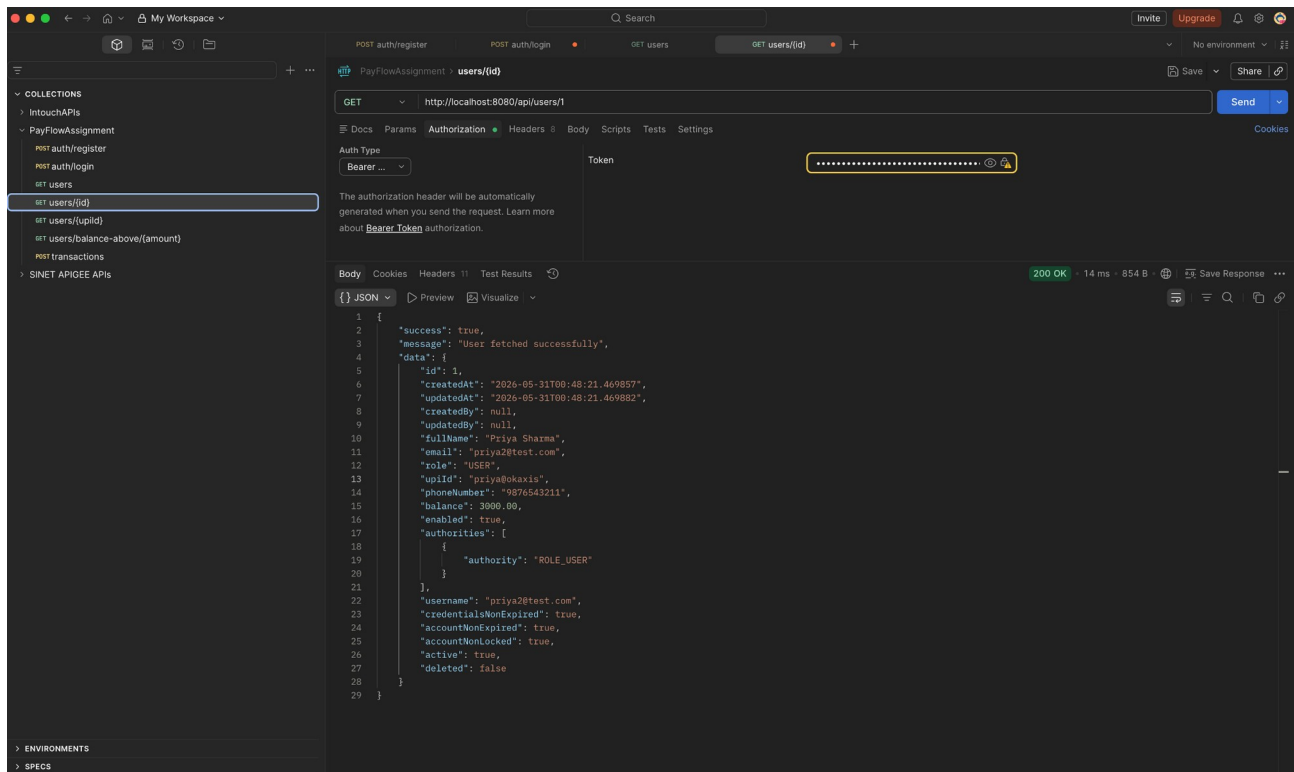
Screenshot: POST /auth/register creates a PayFlow user.



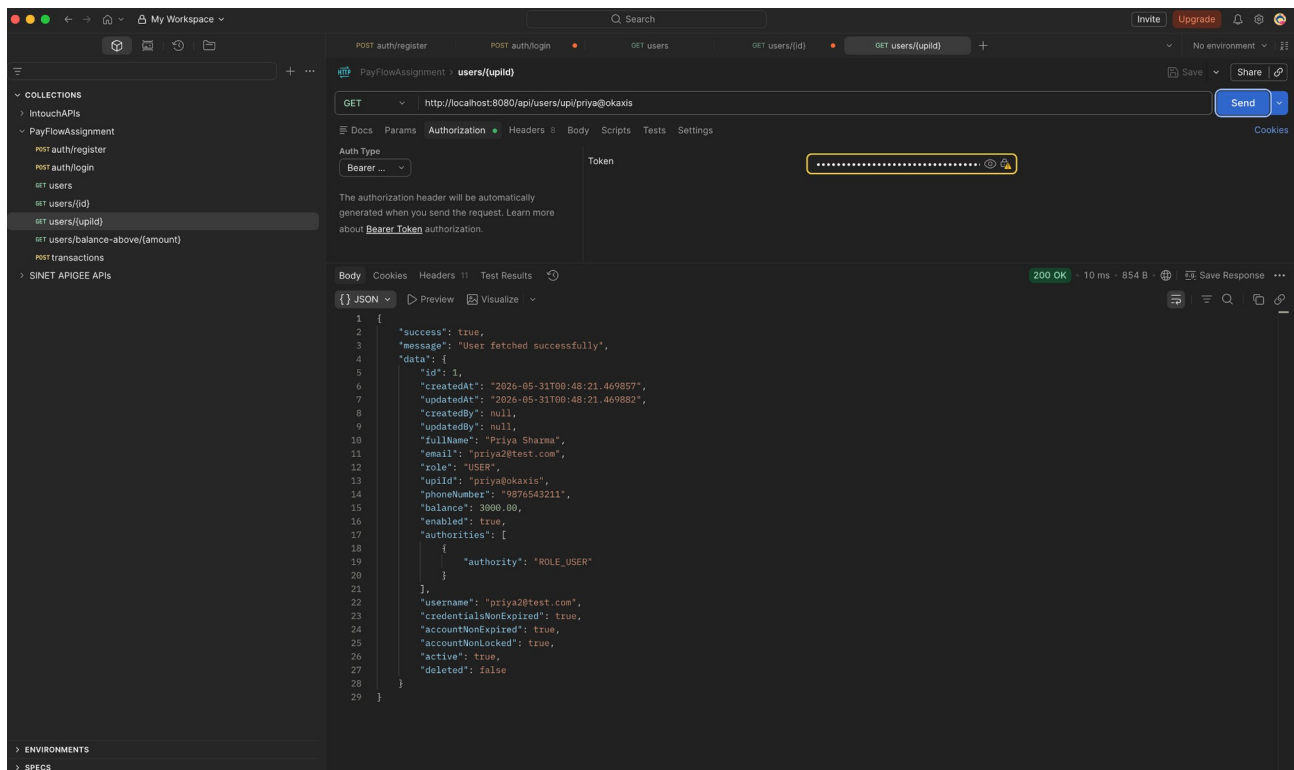
Screenshot: POST /auth/login returns user information and JWT token.



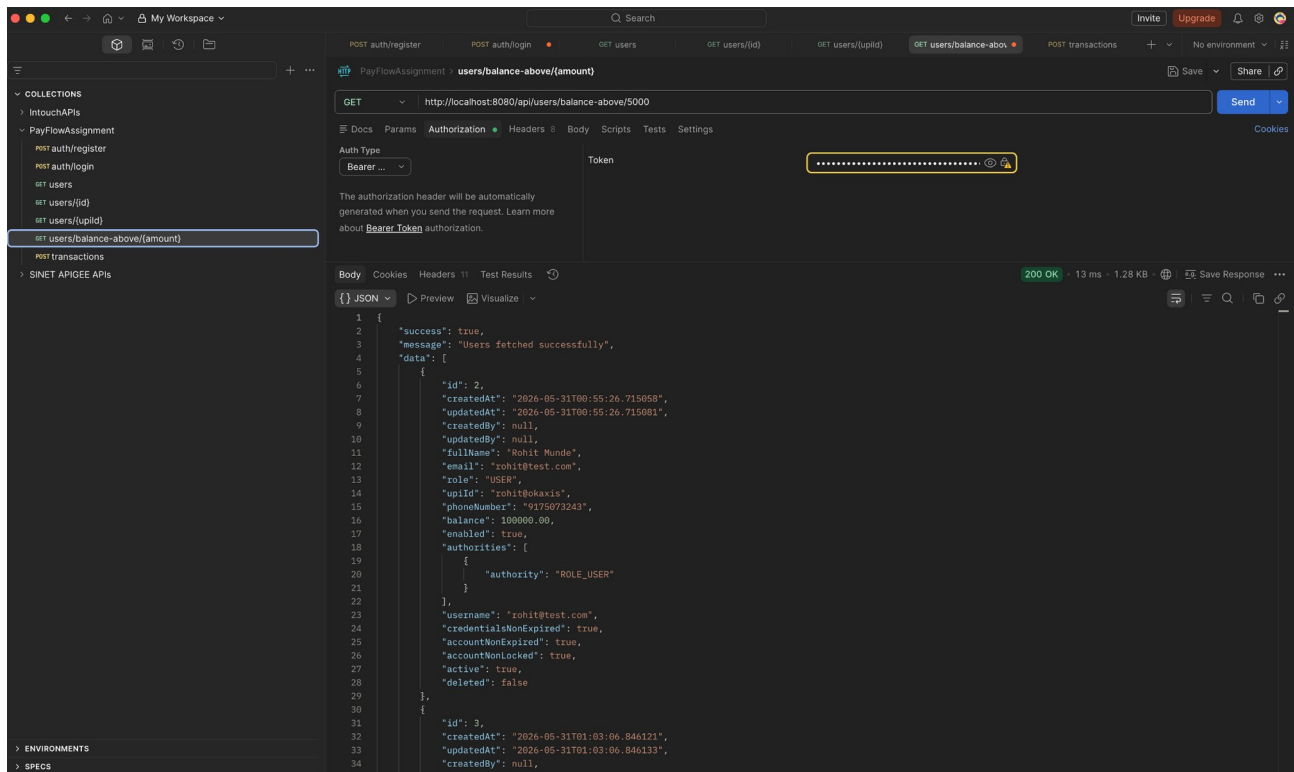
Screenshot: GET /users returns all registered users using Bearer token.



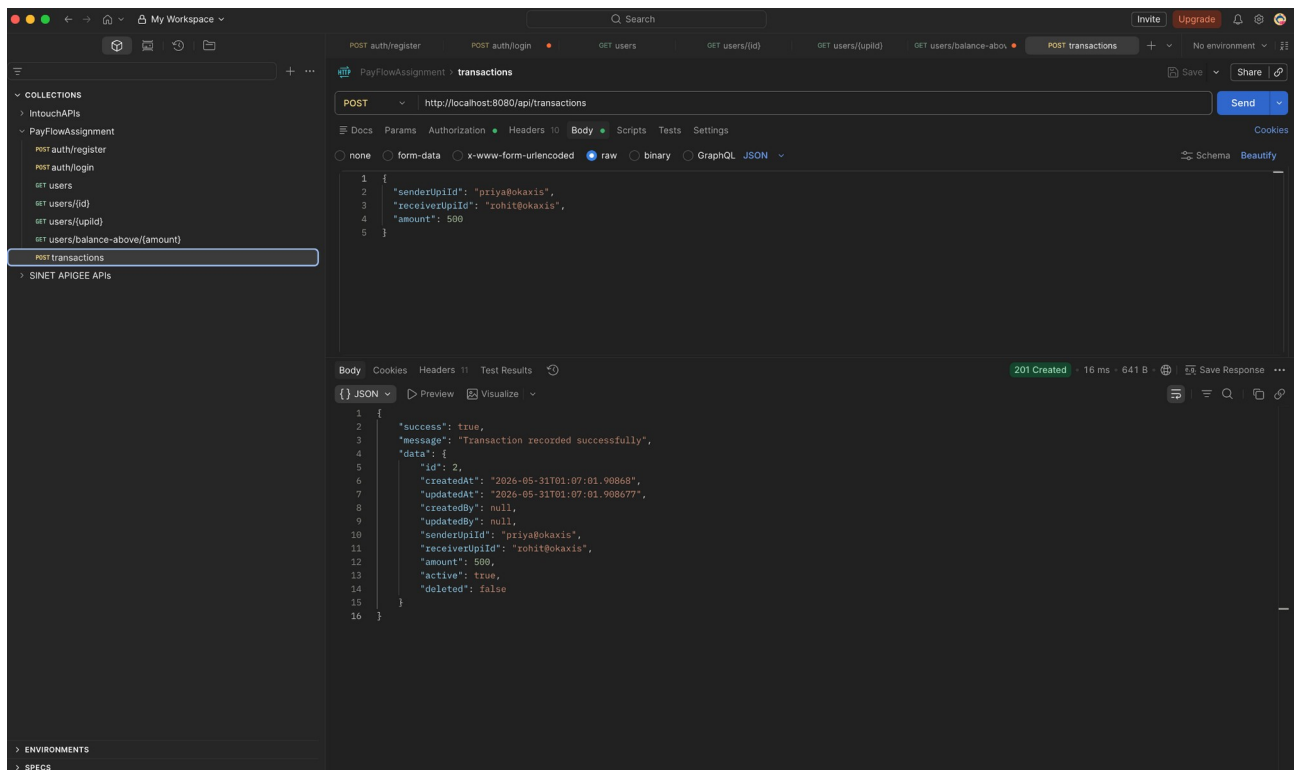
Screenshot: GET /users/{id} returns a user by ID.



Screenshot: GET /users/upi/{upId} returns a user by UPI ID.



*Screenshot: GET /users/balance-above{amount} returns users above the selected balance.*



*Screenshot: POST /transactions records a transaction between two UPI IDs.*

## Curl Commands and Sample Outputs

The same APIs can be tested from the terminal using curl. The protected endpoints require the JWT token returned by /auth/login.



```

# 1. Register user
curl -X POST http://localhost:8080/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "fullName": "Priya Sharma",
  "email": "priya2@test.com",
  "password": "Password@123",
  "upiId": "priya@okaxis",
  "phoneNumber": "9876543211",
  "balance": 3000
}'

# 2. Login
curl -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "priya2@test.com", "password": "Password@123"}'

# Save the returned token in a shell variable
TOKEN="<paste-jwt-token-here>"

# 3. List all users
curl -X GET http://localhost:8080/api/users \
-H "Authorization: Bearer $TOKEN"

# 4. Get user by ID
curl -X GET http://localhost:8080/api/users/1 \
-H "Authorization: Bearer $TOKEN"

# 5. Get user by UPI ID
curl -X GET http://localhost:8080/api/users/upi/priya@okaxis \
-H "Authorization: Bearer $TOKEN"

# 6. Custom query: users with balance above 5000
curl -X GET http://localhost:8080/api/users/balance-above/5000 \
-H "Authorization: Bearer $TOKEN"

# 7. Record transaction
curl -X POST http://localhost:8080/api/transactions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TOKEN" \
-d '{
  "senderUpiId": "priya@okaxis",
  "receiverUpiId": "rohit@okaxis",
  "amount": 500
}'

```

Sample transaction response:

```

{
  "success": true,
  "message": "Transaction recorded successfully",
  "data": {
    "id": 2,
    "senderUpiId": "priya@okaxis",
    "receiverUpiId": "rohit@okaxis",
    "amount": 500,
    "active": true,
    "deleted": false
  }
}

```

## Demonstration of @RequestBody

In the controller, `@RequestBody` tells Spring MVC to read the HTTP request body and convert the incoming JSON into a Java DTO or entity object. With `@RequestBody`, Jackson maps fields such as `fullName`, `email`, `upiId`, `phoneNumber`, and `balance` from JSON into the Java object. Without `@RequestBody`, Spring does not read the JSON body for that parameter; it tries to bind from request parameters or form fields, so the object fields remain null. That is why a POST

request containing JSON works correctly with `@RequestBody` but results in null fields when `@RequestBody` is removed from the controller method parameter.

```
With @RequestBody:
@PostMapping("register")
public ResponseEntity<ApiResponse<UserResponseDto>> createUser(@Valid @RequestBody UserDto user) { ... }
```

Example object inside controller:

```
fullName = "Priya Sharma"
email = "priya2@test.com"
upiId = "priya@okaxis"
phoneNumber = "9876543211"
balance = 3000
```

Without `@RequestBody`:

```
@PostMapping("register")
public ResponseEntity<ApiResponse<UserResponseDto>> createUser(@Valid UserDto user) { ... }
```

Example object inside controller:

```
fullName = null
email = null
upiId = null
phoneNumber = null
balance = null
```

## Task 6 - Custom Query

The custom UPI lookup is implemented using a derived method name in `UserRepository` and exposed through `GET /users/upi/{upild}`. A second custom query is implemented using JPQL with `@Query` to return users whose balance is above a given amount.

```
@Query("SELECT u FROM User u WHERE u.balance > :amount")
List<User> findUsersWithBalanceGreaterThan(BigDecimal amount);
```

| Approach                      | Example                                                        | When to Use                                                                                                                           |
|-------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Derived method name           | <code>findByUpild(String upild)</code>                         | Best for simple equality/filter queries. Spring Data JPA derives SQL from method names.                                               |
| <code>@Query</code> with JPQL | <code>SELECT u FROM User u WHERE u.balance &gt; :amount</code> | Best for more expressive object-oriented queries using entity and field names instead of table names.                                 |
| Native SQL                    | <code>SELECT * FROM users WHERE balance &gt; ?</code>          | Least preferred unless database-specific features are needed, because it couples code to the database schema and reduces portability. |

## Conceptual Write-Up

### 1. Request lifecycle

When curl or Postman sends a POST request to the application, the request first reaches the embedded Tomcat server and then enters the Spring MVC pipeline. The `DispatcherServlet` acts as the front controller and receives the request. It checks the request path and uses `HandlerMapping` to find the correct controller method, such as `AuthController.createUser`. After that, a `HandlerAdapter` invokes the controller method and passes the converted Java object to it. The controller then calls the service layer, which saves data through the repository.

## 2. Serialisation

When a JSON payload is sent, Jackson is the library that converts JSON into a Java object. Spring MVC uses `HttpMessageConverters` internally, and for JSON it uses Jackson-based conversion. If the JSON key is `upild`, it maps to the Java field `upild`. If the key is `upi_id`, it will not map to `upild` by default unless a naming strategy such as `snake_case` is configured or `@JsonProperty("upi_id")` is added.

## 3. Spring Boot features

The three Spring Boot features are embedded server, auto-configuration, and production-ready defaults. The embedded server is visible because the app runs directly with embedded Tomcat on `localhost:8080` without deploying a WAR file. Auto-configuration is visible in H2, JPA repositories, Hibernate table creation, Spring MVC controllers, and JSON conversion working from dependencies and properties. Production-ready defaults are visible through structured configuration, validation, layered beans, exception handling, and secure JWT-protected APIs.

## 4. Spring vs Spring Boot

If this project used plain Spring instead of Spring Boot, I would need to manually configure the `DispatcherServlet`, web application context, dependency versions, embedded server or external server deployment, JPA entity manager, datasource, transaction manager, and JSON message converters. Spring Boot removes most of that manual setup using starter dependencies and auto-configuration. For PayFlow, this means I can focus on entities, repositories, services, and controllers instead of low-level framework configuration.

## 5. Stateless REST

Stateless means the server does not store conversational state between two HTTP requests. Each request must carry everything required to process it, such as the URL, JSON body, and JWT token for authentication. This matters if PayFlow runs behind a load balancer because any server instance can handle any request independently. Stateless design improves scalability and avoids problems where one server knows something that another server does not.

## 6. Persistence

Transactions are stored in the H2 database instead of an in-memory Java List. If a Java List were used and the application restarted, all transaction records would be lost because the list exists only in application memory. That is unacceptable for a payments application because payment records must remain available for audit, debugging, reconciliation, and user trust. Even in this assignment, using a database demonstrates the correct persistence pattern for financial systems.

## Additional Enhancements Beyond Assignment

- JWT-based authentication protects PayFlow APIs so only authenticated users can access them.
- Global exception handling returns consistent error responses.
- DTO responses prevent password exposure in registration/login responses.
- Validation annotations reject invalid input early.
- Service layer validates sender and receiver UPI IDs before recording a transaction.
- Consistent `ApiResponse` wrapper is used for predictable frontend integration.

## Final Checklist

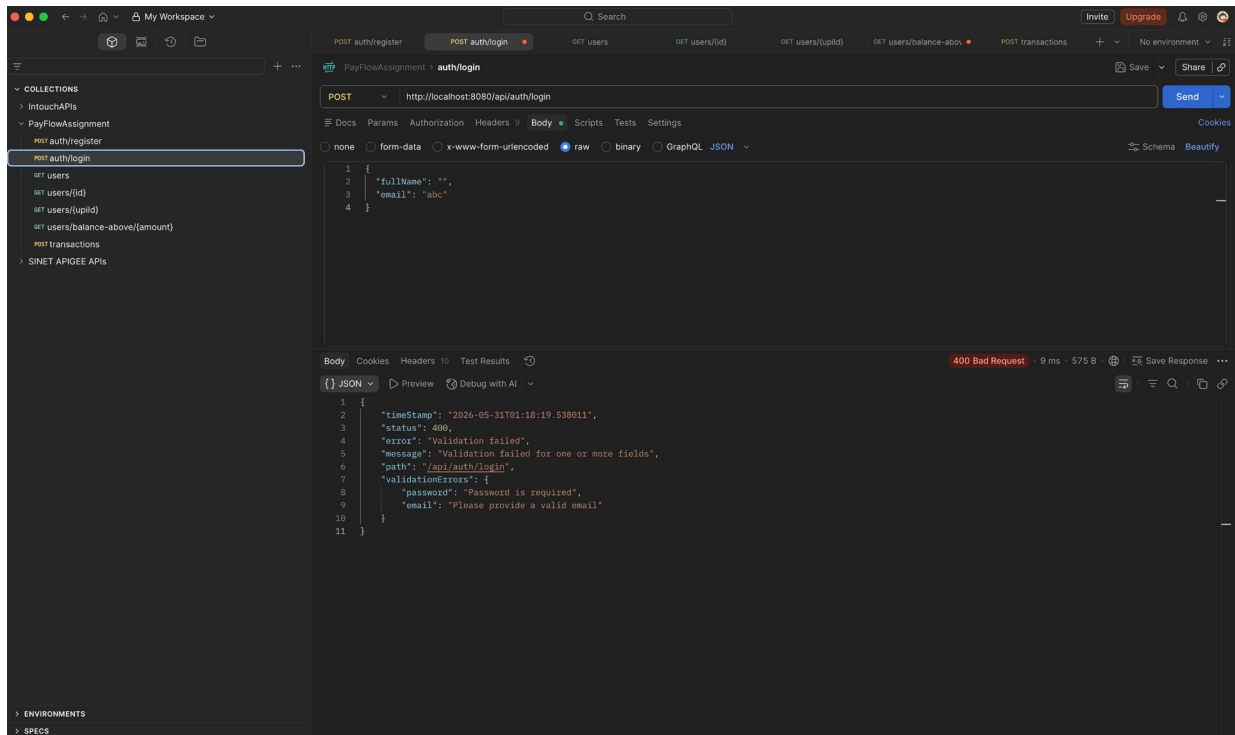
- Complete Spring Boot project pushed to GitHub: <https://github.com/rohit-munde/-PayFlow>
- Application starts cleanly from a fresh run.
- H2 console is enabled and tables are visible.
- Users and transactions are persisted through JPA.
- All required endpoints were tested using Postman/curl-compatible HTTP calls.
- Repository, service, controller, entity, and DTO layers are implemented.
- The write-up contains conceptual answers and screenshots.

# Additional Proof - Validation, Exception Handling, and Security Readiness

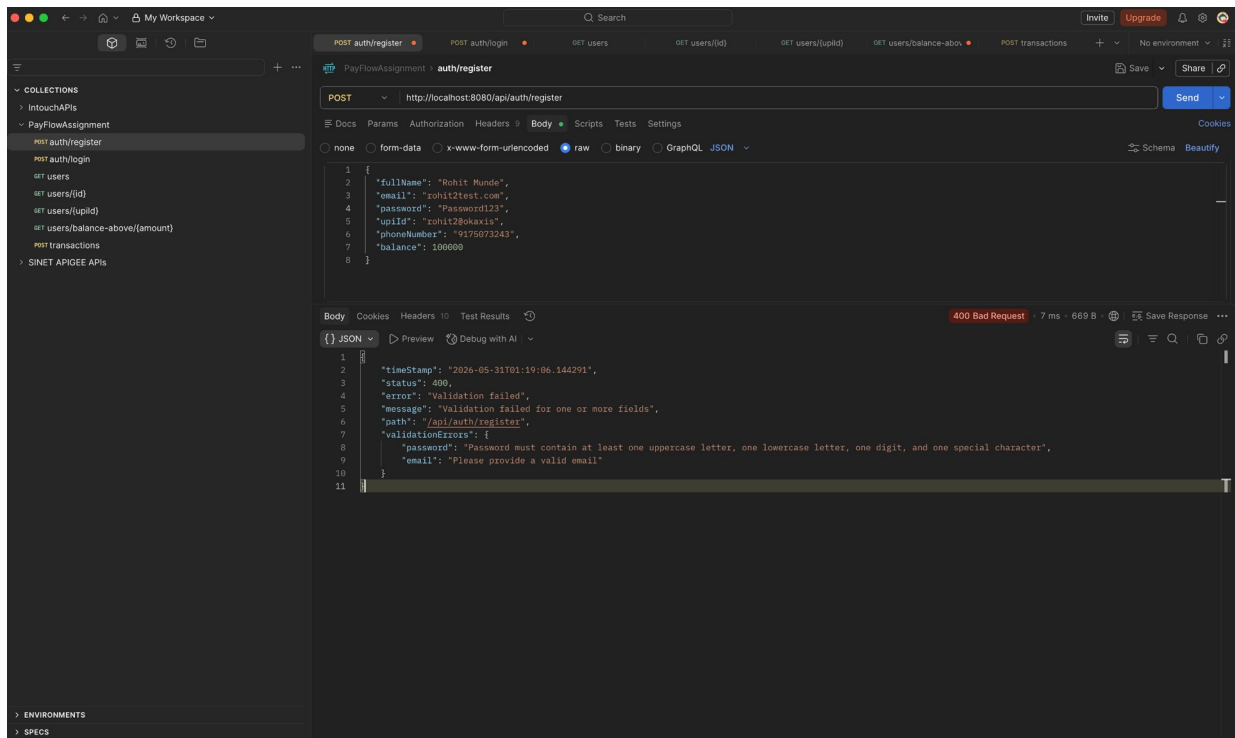
Beyond the core PayFlow assignment requirements, the API includes validation and centralized exception handling. This makes the backend safer for frontend integration because invalid requests and business rule failures return consistent JSON error responses instead of raw server errors.

## Validation Error Handling

The application uses Jakarta Bean Validation annotations such as `@NotBlank`, `@Email`, `@Size`, `@Pattern`, and `@DecimalMin`. When invalid input is submitted, the `GlobalExceptionHandler` captures `MethodArgumentNotValidException` and returns a structured 400 response containing field-level validation errors.



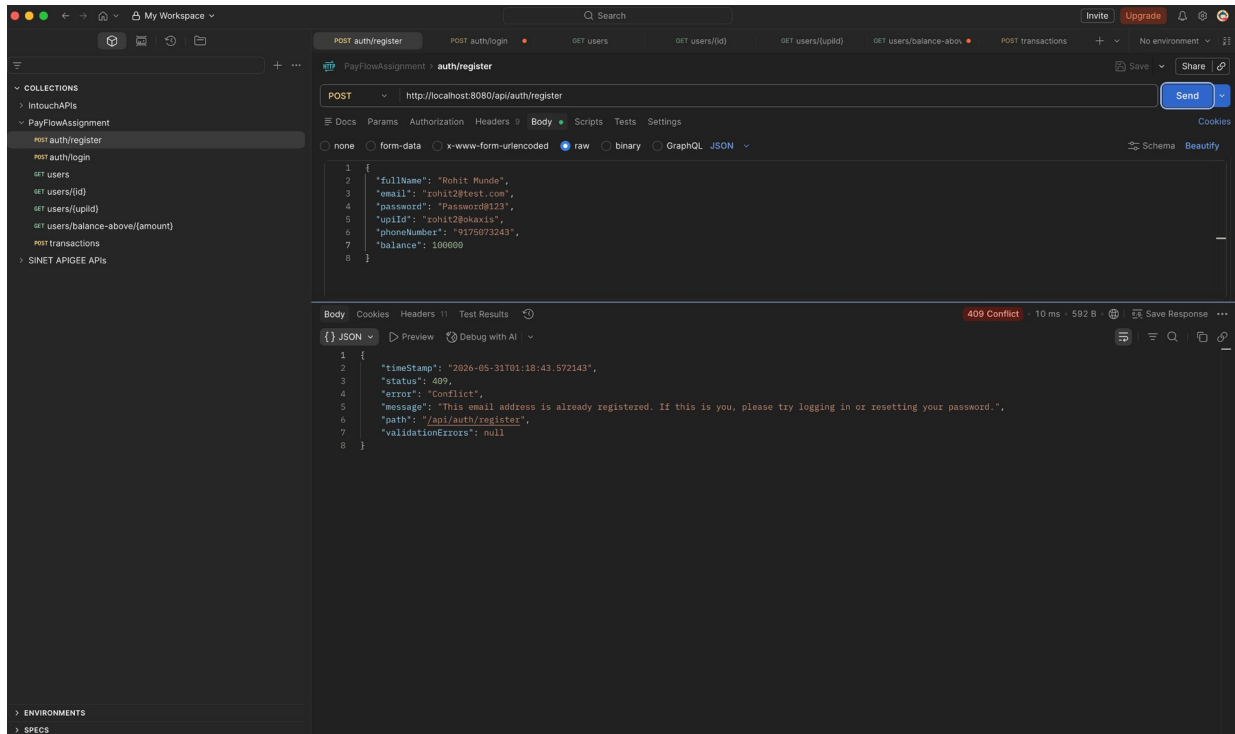
*Validation failure on login request: missing password and invalid email format.*



*Validation failure on registration request: invalid password pattern and invalid email format.*

## Business Exception Handling

The project also handles business-level errors through custom exceptions. For example, duplicate email registration is rejected with a 409 Conflict response instead of allowing inconsistent user data.



*Duplicate email registration handled as a 409 Conflict response.*

## GlobalExceptionHandler Implementation

The centralized exception handler keeps error responses consistent across authentication, validation, and PayFlow APIs. It handles business exceptions, validation exceptions, bad credentials, disabled accounts, and unexpected server errors.

```
@RestControllerAdvice
```

```
public class GlobalExceptionHandler {

    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ApiError> handleBusinessException(BusinessException ex, HttpServletRequest request)
    {
        HttpStatus errorStatus = ex.getHttpErrorStatus();
        ApiError apiError = new ApiError(
            errorStatus.value(),
            errorStatus.getReasonPhrase(),
            ex.getMessage(),
            request.getRequestURI()
        );
        return new ResponseEntity<>(apiError, errorStatus);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidationException(MethodArgumentNotValidException ex,
                                                              HttpServletRequest request) {
        HashMap<String, String> validationErrors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            validationErrors.put(error.getField(), error.getDefaultMessage())
        );

        ApiError apiError = new ApiError(
            HttpStatus.BAD_REQUEST.value(),
            "Validation failed",
            "Validation failed for one or more fields",
            request.getRequestURI(),
            validationErrors
        );
        return new ResponseEntity<>(apiError, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(BadCredentialsException.class)
    public ResponseEntity<ApiError> handleBadCredentialsException(BadCredentialsException ex,
                                                                  HttpServletRequest request) {

        ApiError apiError = new ApiError(
            HttpStatus.UNAUTHORIZED.value(),
            HttpStatus.UNAUTHORIZED.getReasonPhrase(),
```

```

        "Invalid email or password. Please check your credentials and try again.",
        request.getRequestURI()

    );
    return new ResponseEntity<>(apiError, HttpStatus.UNAUTHORIZED);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ApiError> handleGenericException(Exception ex, HttpServletRequest request) {
    ApiError apiError = new ApiError(
        HttpStatus.INTERNAL_SERVER_ERROR.value(),
        HttpStatus.INTERNAL_SERVER_ERROR.getReasonPhrase(),
        "An unexpected error occurred. Please try again later.",
        request.getRequestURI()
    );
    return new ResponseEntity<>(apiError, HttpStatus.INTERNAL_SERVER_ERROR);
}
}

```

## How These Enhancements Support PayFlow

- JWT authentication protects PayFlow APIs so only logged-in users can access wallet and transaction endpoints.
- Validation prevents incomplete or malformed users from being persisted into the H2 database.
- Business exceptions such as duplicate email or duplicate UPI ID are returned as clean API errors.
- The same response structure can be consumed easily by a frontend team.